

Enhancing IoT Applications' Placement in the Fog with Proactive Infeasibility Detection and Repair

Ye Xia
Independent Researcher
Seattle, WA, USA
qazxiaye@gmail.com

Xing Zhang
Independent Researcher
Seattle, WA, USA
jason.xing.zhang@gmail.com

Abstract—Fog computing extends processing and storage resources to the edge of the network by making use of heterogeneous devices. How to efficiently use resources in the fog remains a challenge not only because of the complexity of selecting appropriate hosts for distributed applications, but also due to the lack of guaranteed resource capacities to meet all applications' deployment requests. Application host selection, which is also known as the placement problem, is typically modeled as an optimization problem with hard constraints, which is computationally expensive. Therefore, it is essential to ensure that valid solutions exist (*i.e.*, the problem's feasibility) before passing a placement problem to the optimization module. Otherwise, potential retries in placement decision making can degrade the responsiveness and reliability of the fog infrastructure. This paper presents an infeasibility repair approach to avoid inputting infeasible problems to placement optimization modules. The proposed approach integrates a Machine Learning-based infeasibility detection module with a heuristic algorithm to update the application set under consideration. Multiple datasets¹ representing diverse infrastructures are generated to train and evaluate the proposed model. Experiments show that our model achieves 86% infeasibility detection accuracy, outperforming the state-of-the-art Large Language Model with few-shot prompting. Furthermore, the model demonstrates strong generalizability—it has the capability to suit infrastructures that do not exist in its training data, making it a robust solution for dynamic fog environments. These results highlight the effectiveness of the proposed model in identifying infeasible problems without invoking computationally expensive optimization modules, which can significantly reduce computation time and ensure the system's responsiveness.

Index Terms—Infeasibility Repairing, Machine Learning, Placement, Fog Computing

I. INTRODUCTION

In recent years, fog computing has emerged as a new computational paradigm, extending cloud computing to the edge of the network. By providing resources close to sensors and actuators, it satisfies Internet of Things (IoT) applications that require low response time and data privacy enforcement.

To take advantages of local resources provided by the fog, intelligent decisions are needed for i) determining where to place distributed applications (*i.e.*, selecting hosts for each application) and ii) deciding which applications to deploy (*i.e.*, selecting a set of applications to fulfill with limited resource taken into account). These decisions impact both

resource utilization efficiency and the applications' performance. However, the placement decision-making exhibits the following challenges:

- 1). **Heterogeneity**: a fog infrastructure contains heterogeneous devices located at different network positions. Resource-constrained devices in the infrastructure make it hard to guarantee sufficient resource capacity for all application deployment requests.
- 2). **Dynamicity**: because of applications' arrival / departure and volatile end devices (*e.g.*, devices' mobility / churn), both demand and resources have a constant varying nature. Thus, the placement decision needs to be refreshed continuously.
- 3). **Constraint Diversity**: IoT applications must conform to many kinds of constraints (including hard constraints) to ensure its functionality. These constraints can involve consumable resources (*e.g.*, processing and bandwidth capacities), non-consumable properties (*e.g.*, network latency, privacy), and different entities (*e.g.*, applications' software elements and communication channels).
- 4). **Scalability**: to be reactive to deployment requests, placement decisions must be made time-efficiently. However, known to be NP-hard [1], a placement problem's complexity increases dramatically with the fog's and applications' sizes, which makes it hard to deal with large-scale problems.

Because of limited resources in the fog and the above challenges 1), 2), and 3), it is possible that only a subset of applications can be deployed during a certain time period—a placement problem in the context of fog is possible to be infeasible. Considering the complexity of a placement problem, detecting the infeasibility through solving the problem itself can result in unacceptably long execution time. To deal with this infeasibility in a reactive manner, this work proposes a novel approach based on Machine Learning (ML) models.

The contribution of this paper includes:

- A placement system framework with the capability to detect and repair infeasible placement problems, ensuring reactive and efficient decision-making.
- An ML model for detecting infeasible placement problems, leveraging a novel feature set that effectively represents both the infrastructure and application set. This feature set enables ML models to accurately determine the infeasibility of placement problems.

¹Data and code available at <https://github.com/qazxiaye/InfeasibilityDetection>

- A heuristic algorithm for repairing infeasible placement problems. This algorithm rejects low-priority applications' deployment request and updates the application set for placement.
- A dataset generated based on state-of-the-art placement algorithms [2], [3] and randomly generated infrastructure and applications. The dataset includes approximately 20,000 placement problems across various scales, and has balanced numbers for feasible and infeasible problems.
- A comparative evaluation using state-of-the-art Large Language Model (LLM) as a baseline. As LLMs are found to be effective regressors [4], our proposed infeasibility detection model has been compared to few-shot LLM.

To the best of the authors' knowledge, this is the first work addressing challenges brought by problems' infeasibility in the context of applications' placement in the fog. Experiments show that our proposed model effectively detects infeasibility with up to 86% detection accuracy, outperforming few-shot LLMs significantly. Moreover, with an infrastructure-agnostic design, our model has the capability to transfer its learning from one infrastructure to others—it can effectively detect infeasibility for infrastructures that do not exist in its training data.

The remainder of this paper is organized as follows. [Section II](#) reviews related works. [Section III](#) defines the placement problem and its feasibility / infeasibility. [Section IV](#) details the proposed infeasibility repair approach. [Section V](#) describes the process for generating datasets used in this work. [Section VI](#) presents the evaluation of the proposed ML model. [Section VII](#) concludes and discusses future works.

II. RELATED WORK

A number of studies address the placement problem in fog and cloud computing [2], [3], [5]–[8]. However, all these works assume that the input problems to their proposed algorithms are feasible, without discussing how to handle potential infeasibility.

Infeasibility detection has been studied in the context of linear programming [9]–[11]. Nevertheless, these approaches are not applicable to nonlinear programming problems including the placement problem.

Several prior works have addressed infeasibility repair. For instance, [11], [12] propose methods to repair infeasibility in linear programming problems. Although these approaches aim to minimize changes during repair, they may modify constraints. As a result, they can alter applications' resource requirements and compromise applications' functionality, which makes them unsuitable for IoT application placement. Another work [13] addresses infeasibility in job scheduling, which is more similar to IoT placement. However, this approach is tightly coupled with a genetic algorithm, limiting its adaptability.

ML technique has been extensively explored for solving optimization problems. There are studies leveraging ML models for parameter tuning, solution initialization, and solution evolution [14]. Some studies [15], [16] directly utilizes ML

models for decision-making. However, these work do not guarantee satisfaction of hard constraints—hard constraints are possible to be violated, leaving the infeasibility problem unresolved.

LLMs have also been explored in related fields. [17] proposes a natural language-based system to deal with infeasible problems through interactive conversations. However, such human-involved solutions are unsuitable for time-sensitive problems, such as the ones in the context of fog. [18] develops evaluation framework and studies LLMs' performance in handling tasks with constraints. It is found that state-of-the-art LLMs still face significant challenges in achieving constraint satisfaction.

III. FORMULATION

This section discusses the placement problem's constraints and formulates its feasibility / infeasibility.

A fog infrastructure contains two kinds of devices: i) *fog nodes* (e.g., cloud, edge server), which provide processing and storage resources; ii) *appliances* (e.g., sensor, actuator), which provide sensing / actuating services. Only fog nodes can be used as hosting devices. Appliances do not provide resources to host applications. A fog infrastructure also consists of links, which connect devices and provide network resources.

An IoT application is composed of components, bindings, and appliances. A *component* is a software element that can be executed on one fog node. A *binding* is a communication channel that connects a couple of components or a component and an appliance.

<i>Infra</i>	a fog infrastructure
<i>node_i</i>	a fog node of <i>Infra</i>
<i>node_i.CPU</i>	<i>node_i</i> 's available CPU capacity
<i>node_i.RAM</i>	<i>node_i</i> 's available RAM capacity
<i>node_i.DISK</i>	<i>node_i</i> 's available DISK capacity
<i>appliance_i</i>	an appliance of <i>Infra</i>
<i>link_i</i>	a link of <i>Infra</i>
<i>link_i.LAT</i>	<i>link_i</i> 's network latency
<i>link_i.BW</i>	<i>link_i</i> 's available bandwidth capacity
<i>Apps</i>	a set of applications to place
<i>comp_i</i>	a component of an application in <i>Apps</i>
<i>comp_i.ReqCPU</i>	<i>comp_i</i> 's CPU requirement
<i>comp_i.ReqRAM</i>	<i>comp_i</i> 's RAM requirement
<i>comp_i.ReqDISK</i>	<i>comp_i</i> 's DISK requirement
<i>bind_i</i>	a binding of an application in <i>Apps</i>
<i>bind_i.ReqLAT</i>	<i>bind_i</i> 's latency requirement
<i>bind_i.ReqBW</i>	<i>bind_i</i> 's bandwidth requirement
<i>app_i</i>	an application in <i>Apps</i>
<i>app_i.components</i>	all components of <i>app_i</i>
<i>app_i.appliances</i>	all appliances of <i>app_i</i>
<i>app_i.priority</i>	<i>app_i</i> 's priority

TABLE I: Summary of Notations.

As summarized in [Table I](#), this work's model characterizes each component *comp* with *ReqCPU*, *ReqRAM*, and *ReqDISK*, which respectively indicate CPU, RAM, and DISK resources that *comp* consumes. Bindings are characterized with requirements as well. *bind_i.ReqBW* designates *bind_i*'s bandwidth requirement. *bind_i.ReqLAT* indicates the maximal network latency that *bind_i* can accept.

A placement problem is feasible if there exists a placement that satisfies the following four constraints. Otherwise, it is infeasible.

- each component is placed to a node;
- total CPU / RAM / DISK requirement in a fog node does not exceed its capacity;
- total bandwidth requirement for a link does not exceed its capacity;
- a binding's latency does not exceed its requirement.

IV. METHODOLOGY

The proposed infeasibility repair approach identifies a set of applications that can be deployed using the available resources in the fog infrastructure. As shown in Figure 1, the approach consists of two steps:

- **Infeasibility Detection**, which detects infeasible problems based on available resources in the infrastructure and the current application deployment requests.
- **Application Selection**, which updates the application set in deployment requests based on infeasibility detection outputs.

After the infeasibility repair process, the final application set will be an input of the placement optimizer for making placement decisions. Details for Infeasibility Detection and Application Selection are provided in Section IV-A and Section IV-B, respectively.

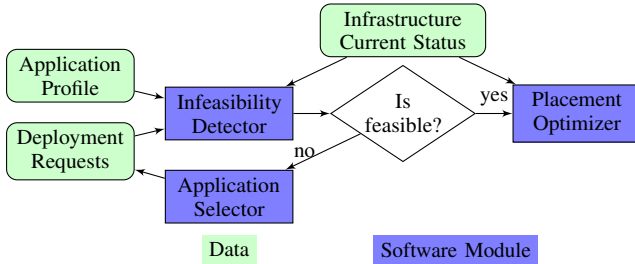


Fig. 1: Placement Decision Making System Framework.

A. ML-based Infeasibility Detection

Determining infeasibility in placement problems is challenging due to the interdependent resource consumption of multiple applications—the applications share a common infrastructure with limited resources. Straightforward cases, such as a component requiring excessively high CPU, can be addressed with rule-based approaches. However, this work focuses on complex scenarios, where infeasibility arises from the cumulative resource demands of multiple applications. This section discusses the causal variables used for infeasibility detection and details the design of a supervised ML model.

1) *Causal Variables for Infeasibility Detection*: To detect placement problems' infeasibility, it is essential to compare resource capacities of the infrastructure with the resource demands of application deployment requests. Key causal variables include: fog node count, application count, component count, total resource capacity, and total resource requirement.

Since IoT applications are bound to appliances, these variables must be assessed for specific network areas. We define a node n 's sub-net as the set of devices under n 's coverage in the network. For example, in Figure 2, the sub-net of gateway #2 includes gateway #2, end fog node #1, end fog node #2, and appliance #2. Similarly, a sub-net's applications are those bound to its appliances. For instance, applications of gateway #2's sub-net are the ones bound to appliance #2. Applications bound to appliances across multiple sub-nets are included in all relevant sub-nets. Based on the above definitions, causal variables for a sub-net n can provide critical insights for estimating n 's criticality—how difficult it is to satisfy n 's applications with available resources in n .

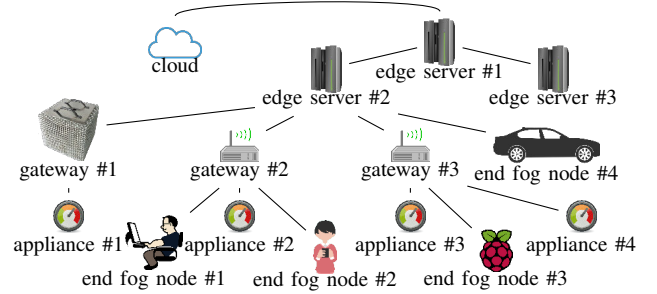


Fig. 2: Infrastructure Example (containing a cloud, three edge servers, three gateways, four end fog nodes, and four appliances).

2) *Identifying Critical Sub-nets*: In Figure 2, the infrastructure consists of four network layers: Gateway layer, Local edge server layer (i.e., edge servers #2 and #3), Regional edge server layer (i.e., edge server #1), and Cloud layer. Each layer contains multiple sub-nets (e.g., sub-nets of gateway #1, #2, and #3 in the gateway layer). To make the ML model infrastructure-agnostic, individual sub-net details are excluded from the feature set. Instead, statistical information from the most critical sub-nets in each layer is used. Critical sub-nets are identified based on the following criteria:

- Node count per application: the node count divided by the application count in the sub-net $\mathcal{N}_i$

$$\frac{|\text{node} \in \mathcal{N}_i|}{|\text{app} \in \mathcal{N}_i|}$$

- Node count per component: the node count divided by the total component count of the applications in sub-net $\mathcal{N}_i$

$$\frac{|\text{node} \in \mathcal{N}_i|}{|\text{comp} \in \mathcal{N}_i|}$$

- Per-application upward bandwidth: taking gateway #2 in Figure 2 as an example, this criteria is the bandwidth between gateway #2 and edge server #2 (i.e., the bandwidth from the considered sub-net to the higher network layer) divided by count of applications bound to appliances in gateway #2's sub-net;

- Capacity-to-demand ratio: For example,

$$\text{the ratio for CPU} = \frac{\sum_{\text{node} \in \mathcal{N}_i} \text{node.CPU}}{\sum_{\text{comp} \in \mathcal{N}_i} \text{comp.ReqCPU}},$$

similar for RAM and DISK.

A lower criteria value indicates a more critical sub-net. Criteria iv) covers different resource types — CPU, RAM, and DISK, but only the most critical resource type (*i.e.*, the one with the lowest ratio) is considered.

3) *Feature Set for the ML Model*: For each network layer, the ML model includes the following features:

- i) the lowest *node count per application*;
- ii) the lowest *node count per component*;
- iii) the lowest *per-application upward bandwidth*;
- iv) the lowest *capacity-to-demand ratio*.

In typical infrastructures with four network layers, the model's feature set contains 16 features (4 per layer). For example, if the node count per application is 0.5, 1.5, and 2.5 for sub-net *gateway #1*, *gateway #2*, and *gateway #3*, respectively, the model selects 0.5 as the feature value. Values for feature i), ii), and iii) are selected by comparing all sub-nets in the considered network layer, while the values for feature 4) is selected based on comparison among sub-nets as well as different resource types—CPU, RAM, and DISK. This design avoids assumptions about a specific infrastructure topology, ensuring our model's generality for dynamic and diverse infrastructures.

4) *ML Model Implementation*: The proposed model leverages XGBoost [19], a regression framework known for its efficiency. Its fast execution time supports reactive infeasibility detection. Hyper parameters used in our model are given in Table II.

Hyper Parameter	Value
objective	reg:logistic
tree_method	exact
learning_rate	0.1
max_depth	6

TABLE II: Hyper Parameters.

With the above hyper parameters, the model's output will be continuous floating numbers, which indicates the likelihood of the problem's infeasibility. This value is converted to binary using a threshold. A lower threshold makes the model more aggressive—the model tends to classify more problems as infeasible. In this work, we simply set this threshold to 0.5.

B. Heuristic-based Application Selection

When an infeasible placement problem is detected, the system repairs it by rejecting certain applications' deployment requests. This process selects applications based on their assigned priority² and the criticality of the sub-nets they are associated with.

We propose a heuristic algorithm (see Algorithm 1) to filter out low-priority applications bound to the most critical sub-nets. The algorithm uses two parameters:

- *candidate_count_per_round*: The number of candidate applications considered for filtering in each round.
- *max_number_of_app*: The maximum number of applications that can be rejected overall.

These parameters can be configured based on the infrastructure's scale and specific use cases.

Line 2-5 selects candidate applications for rejection from applications of each network layer's critical sub-nets. For each network layer and each criticality criterion, the algorithm identifies the most critical sub-net (`GetMostCriticalSubNet`, line 4) based on the criteria discussed in Section IV-A. It is worth noting the criteria value is calculated during the Infeasibility Detection process, which can be directly reused in this Application Selection step. In line 5, candidate applications are selected based on their priority. Line 6 performs additional round of selection to ensure the total number of filtered applications does not exceed *max_number_of_app*.

Algorithm 1: SelectApps

```

Input: candidate_count_per_round, max_number_of_app
1 candidate_apps = {};
2 for each layer  $\in$  network_layers do
3   for each criteria  $\in$  criteria_set do
4     sub_net  $\leftarrow$  GetMostCriticalSubNet(layer, criteria);
5     candidate_apps += GetLowestPriorityApps(sub_net,
        candidate_count_per_round);
6 return SelectLowPriorityApps(candidate_apps, max_number_of_app);

```

As shown in Figure 1, the application selection module iteratively filters out low-priority applications until the placement problem becomes feasible. This process ensures that only valid inputs are passed to the computation-expensive placement optimization step.

V. DATASETS

This section describes the dataset used to train and test the ML model proposed in Section IV-A. The following subsections provide details on the dataset generation process, including the configuration of fog infrastructure and applications, the methodology for determining the infeasibility of placement problems, and the datasets generated for various infrastructure scales.

A. Fog Infrastructure

Fog infrastructures are inherently heterogeneous. Even among nodes of the same type (*e.g.*, two edge servers), their capacities can vary significantly. To accurately reflect this characteristic, the dataset assigns each fog node a randomly distributed capacity within a range specific to its type, as detailed in Table III.

Device Type	CPU (GFlops)	RAM (GB)	DISK (GB)
cloud	infinite	infinite	infinite
edge server	0 ~ 100	0 ~ 500	0 ~ 5000
gateway	0 ~ 8	0 ~ 10	0 ~ 500
end fog node	0 ~ 2	0 ~ 4	0 ~ 200

TABLE III: Capacity Ranges of Each Fog Node Type.

²An application can be associated with a priority level based on their Service Level Agreement (SLA), see Table I.

Similarly, network link resources also follow uniform distributions, with the ranges for network latency and available bandwidth for each link type detailed in Table IV.

Link Type	LAT(ms)	BW(Mbps)
cloud – edge server	30 ~ 100	0 ~ 1000
edge server – edge server	3 ~ 10	0 ~ 1000
gateway – edge server	1 ~ 20	0 ~ 100
end fog node – edge server	10 ~ 25	0 ~ 100
end fog node – gateway appliance – gateway	1 ~ 5	0 ~ 1000

TABLE IV: Capacity Ranges of Each Link Type.

B. Data Stream Processing Application

Data Stream Processing (DSP) applications are widely used across various domains, such as smart transport / building / home, etc. In this work, We choose DSP applications as the placement use case. A DSP application can be represented as a directed acyclic graph, which contains three kinds of vertexes: data source, operator, and data consumer. A *data source* continuously generates a data stream; an *operator* receives and processes incoming streams, and then produces outgoing streams transferred to following operators or data consumers; a *data consumer* only receives and processes incoming streams.

An example DSP application is illustrated in Figure 3. The application maintains a database storing real-time traffic load information, which is used for enabling smart traffic lights and route planning. Data sources—traffic sensors sense traffic loads, whose raw data are filtered and aggregated by operators—filters and aggregators, respectively. Finally, the data consumer—knowledge base updates stored information.

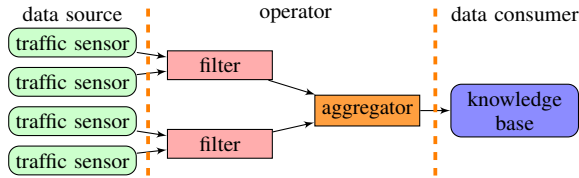


Fig. 3: DSP Application Example—Traffic Knowledge Base (containing four road traffic sensors as data sources, three operators of two types, and one data consumer).

In our dataset, each generated DSP application contains 1~10 data sources, 1~5 operator types, and 1~10 data consumers. Each operator type is in charge of one kind of analytical tasks (*i.e.*, processing a particular type of incoming streams), and has a set of operators to distribute these tasks. Each operator type has 1~10 components as its operators. Data sources, data consumers and operators communicate through bindings. To ensure proper execution of DSP applications, each component type and binding type has specific resource requirements, as detailed in Table V.

component resource requirements			binding resource requirements	
ReqCPU	0.1 ~ 1	GFlops	ReqLAT	25 ~ 50 ms
ReqRAM	0.1 ~ 1	GB	ReqBW	0.01 ~ 1 MBps
ReqDISK	0 ~ 2	GB		

TABLE V: Resource Requirements of DSP Applications.

C. Dataset Generation Process

We generate three groups of datasets: Small (S), Medium (M), and Large (L). The device counts and total number of generated placement problems for each group are summarized in Table VI. Resource capacity, links, and applications are randomly generated according to distribution described in previous sections. To create a placement problems for a given infrastructure, applications are added incrementally until the problem becomes infeasible. When the problem is found to be infeasible, we remove the last application added to the problem, and continue the attempts to add other new applications for creating new problems. The infeasibility determination is based on state-of-the-art placement algorithms proposed in [2], [3].

Dataset	Device Count				
	Cloud	Edge Server	Gateway	End Fog Node	Appliance
Small	1	3	3	20	40
Medium	1	3	20	200	400
Large	1	5	30	300	500

Dataset	Total Problem Count = Infra Count × Problem Count per Infra
Small (S)	9000 = 360 × 25
Medium (M)	6000 = 100 × 60
Large (L)	5000 = 50 × 100

TABLE VI: Dataset Generation Setup.

The setup for *Problem Count per Infra*, as described in Table VI, specifies the maximum number of applications that can be added to a problem. This configuration ensures a balanced distribution of feasible and infeasible problems for each generated infrastructure.

VI. EVALUATION

This section evaluates the ML model proposed in Section IV-A. Section VI-A details the setup for the experiments. Section VI-B discusses the experimental results.

A. Experiment Setup

We evaluate the model through three experiments. Based on datasets generated for Small, Medium, and Large infrastructures (*i.e.*, S, M, and L discussed in Table VI), training and testing datasets used in each experiment are listed in Table VII.

Experiment	Training Dataset	Testing Dataset
All-Infra	80% of (S, M, and L)	20% of (S, M, and L)
Infra-Specific	80% of M	20% of M
Infra-Agnostic	S and M	L

TABLE VII: Training and Testing Datasets.

Because LLMs are capable to perform regression tasks when given in-context examples [4], we use LLM with few shot as a baseline to compare with the proposed model. The LLM result is based on Claude 3 Sonnet. The prompt used in the experiment is similar to the ones used in [4]:

The task is to provide your best estimate for "Output" based on the given examples and features. Please provide that and only that by responding a single character (either 0 or 1) without any additional text.

Feature 0: 2.06
Output: 0

Feature 0: 0.64
Output: 1

Feature 0: 1.62
Output:

Due to the limitation on the maximum number of tokens, the prompt for each LLM call in this experiment is generated using 1,000 randomly selected examples.

B. Results and Discussion

Table VIII summarizes the accuracy, recall, and precision of each model across the experiments. The results demonstrate the effectiveness of the proposed model in detecting infeasibility:

- When trained and tested on data from mixed infrastructure scales, the model achieves an accuracy of approximately 80.6%.
- When both the training and testing data come from a specific infrastructure scale (*i.e.*, Medium in this experiment), the accuracy increases to 86%. This is expected, as the training data more closely reflects the scenarios where the model is evaluated.
- When the model is tested on data from an infrastructure scale that the model has never encountered during its training, the accuracy is still up to 77.6%. This indicates that the insights the model learned during training are transferable to other infrastructure scales, which highlights the generic nature of our approach and its potential to handle problems across diverse infrastructures.
- The imbalance between recall and precision suggests room for further improvement. The consistently higher recall compared to precision indicates that the model tends to classify more problems as infeasible. Adjusting the threshold parameter, as discussed in Section IV-A, could help further optimize the accuracy.

Table VIII also compares the proposed model with LLM-based baseline. Overall, the proposed model outperforms the LLM by over 1,000 basis points across all experiments. Additionally, the proposed model offers significant advantages in execution speed. As a lightweight model, it can handle

Experiment	Proposed Model		
	Accuracy	Recall	Precision
All-Infra	80.6%	86.0%	72.6%
Infra-Specific	86.2%	91.5%	77.9%
Infra-Agnostic	77.6%	86.0%	72.1%

Experiment	Baseline LLM		
	Accuracy	Recall	Precision
All-Infra	69.1%	92.9%	57.9%
Infra-Specific	60.6%	98.9%	50.4%
Infra-Agnostic	46.7%	99.4%	45.4%

TABLE VIII: Infeasibility Detection Accuracy of Different Models.

thousands of infeasibility detection tasks almost instantly (*i.e.*, within 1 second).

Interestingly, contrary to the observation in [4] where LLMs outperformed traditional supervised models in certain scenarios, the LLM significantly underperformed the proposed XGBoost model in this study. This discrepancy can be attributed to the following factors:

- **Problem Complexity:** Unlike the single-feature use cases explored in [4], this experiment involves multiple features, making the problem more complex for the LLM.
- **Feature Characteristics:** Individual features in this study lack strong correlations with the infeasibility detection target, favoring a tree-based model like XGBoost, which excels at handling such scenarios.

Additionally, the extreme imbalance between recall and precision in the LLM results highlights its potential for further enhancement.

Compared to approaches that rely on optimization models to determine infeasibility (*i.e.*, identifying infeasibility when the optimization model fails to find a solution or exceeds a timeout), our proposed model significantly enhances system reactivity. For problems studied in this work, our model completes infeasibility detection within one second, whereas optimization-based approaches may take several minutes to fail or time out.

VII. CONCLUSION AND FUTURE WORK

This paper tackles the challenge of potential infeasibility in optimization problems for placement decision-making in fog computing. We propose a computationally efficient infeasibility repair approach, which ensures reactive and reliable placement decision-making. The proposed approach integrates an ML-based infeasibility detection module with a heuristic-based application selection module. Additionally, this work presents multiple datasets (open-sourced at <https://github.com/qazxiaye/InfeasibilityDetection>), which are used in the evaluation of the proposed ML model.

The evaluation demonstrates the following key findings: i) the proposed model effectively detects infeasibility in placement problems, achieving up to 86% accuracy; ii) the model generalizes well and has the capability to suit different infrastructures, including those not seen during the model training; iii) infrastructure-specific training data, *i.e.*, training

data derived from infrastructures similar to the target infrastructure, further enhances the model's performance; iv) our model outperforms state-of-the-art LLM in both execution time and accuracy.

For future work, to further improve the detection accuracy, it is worth experimenting with larger datasets and more advanced ML models, such as deep learning model, or further improving LLM's performance. There also exists opportunities to improve the system by taking predicted information into account in the application selection module. For example, taking predicted application deployment requests into account when selecting applications, which could help reduce application outages and migration costs.

REFERENCES

- [1] Antonio Brogi, Stefano Forti, and Ahmad Ibrahim. How to best deploy your fog applications, probably. In *International Conference on Edge and Fog Computing*, 2017.
- [2] Ye Xia, Xavier Etchevers, Loïc Letondeur, Lebre Adrien, Thierry Coupaye, and Frédéric Desprez. Combining Heuristics to Optimize and Scale the Placement of IoT Applications in the Fog. In *IEEE/ACM International Conference on Utility and Cloud Computing*. IEEE/ACM, 2018.
- [3] Ye Xia, Xavier Etchevers, Loïc Letondeur, Thierry Coupaye, and Frédéric Desprez. Combining Hardware Nodes and Software Components Ordering-based Heuristics for Optimizing the Placement of Distributed IoT Applications in the Fog. In *The 33rd ACM/SIGAPP Symposium On Applied Computing*. ACM, 2018.
- [4] Robert Vacareanu, Vlad-Andrei Negru, Vasile Suciu, and Mihai Surdeanu. From words to numbers: Your large language model is secretly a capable regressor when given in-context examples. *arXiv preprint arXiv:2404.07544*, 2024.
- [5] Maryam Mirzapour-Moshizi, Vahid Sattari-Naeini, and Amir Sabbagh Molahosseini. Application placement in fog-cum-cloud environment based on a low latency policy-making framework. *Cluster Computing*, 27(1):199–217, 2024.
- [6] Anna-Valentini Michailidou, Christos Bellas, and Anastasios Gounaris. Optimizing task allocation in multi-query edge analytics. *Cluster Computing*, pages 1–18, 2024.
- [7] Juan Luis Herrera, Javier Berrocal, Stefano Forti, Antonio Brogi, and Juan Manuel Murillo. Multi-layered continuous reasoning for cloud-iot application management. *IEEE Transactions on Services Computing*, 2024.
- [8] Ye Xia, Xavier Etchevers, Loïc Letondeur, Thierry Coupaye, and Frédéric Desprez. Optimizing cloud application scheduling: A dual-stage heuristic approach. In *2024 9th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, pages 134–140. IEEE, 2024.
- [9] Jisun Park and Ernest K Ryu. Accelerated infeasibility detection of constrained optimization and fixed-point iterations. In *International Conference on Machine Learning*, pages 27294–27345. PMLR, 2023.
- [10] David Applegate, Mateo Díaz, Haihao Lu, and Miles Lubin. Infeasibility detection with primal-dual hybrid gradient for large-scale linear programming. *SIAM Journal on Optimization*, 34(1):459–484, 2024.
- [11] Sitoshna Jatty, Niharika Singh, Ignacio E Grossmann, Leonardo Salsano de Assis, Christos Galanopoulos, Pablo Garcia-Herreros, Bianca Springub, and Nga Tran. Diagnosis of linear programming supply chain optimization models: Detecting infeasibilities and minimizing changes for new parameter values. *Computers & Chemical Engineering*, 171:108139, 2023.
- [12] A fuzzy method to repair infeasibility in linearly constrained problems. *Fuzzy Sets and Systems*, 122(2):237–243, 2001.
- [13] Raul Mencia, Carlos Mencia, and Ramiro Varela. Efficient repairs of infeasible job shop problems by evolutionary algorithms. *Engineering Applications of Artificial Intelligence*, 104:104368, 2021.
- [14] Maryam Karimi-Mamaghan, Mehrdad Mohammadi, Patrick Meyer, Amir Mohammad Karimi-Mamaghan, and El-Ghazali Talbi. Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. *European Journal of Operational Research*, 296(2):393–422, 2022.
- [15] Priya L Donti, David Rolnick, and J Zico Kolter. Dc3: A learning method for optimization with hard constraints. *arXiv preprint arXiv:2104.12225*, 2021.
- [16] Meiyi Li, Soheil Kolouri, and Javad Mohammadi. Learning to solve optimization problems with hard linear constraints. *IEEE Access*, 11:59995–60004, 2023.
- [17] Hao Chen, Gonzalo E Constante-Flores, and Can Li. Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*, 62(4):573–587, 2024.
- [18] Lior Madmoni, Amir Zait, Ilia Labzovsky, and Danny Karmon. The ability of large language models to evaluate constraint-satisfaction in agent responses to open-ended requests. *arXiv preprint arXiv:2409.14371*, 2024.
- [19] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.